Contents

BASIC WEB VULNERABILITY ASSESSMENT **1**
Internship Details 1
Table of Contents 2
1. Introduction 2
2. Objectives 2
3. Scope and Authorization 3
Included activities 3
Excluded activities 4
4. Tools and Environment 4
The following tools and environment details were recorded in the supplied terminal output. 4
Finding F-02: Publicly Accessible robots.txt Discloses a Path for Further Investigation 8
Finding F-03: Potential Exposure of Sensitive Files and Administrative Resources 8
Informational observation: Access-Control-Allow-Origin Header . 10
Informational observation: Uncommon X-Recruiting Header . . . 10
8. Risk and Impact Analysis 11
The assessment identified one confirmed low-severity configuration observation, one confirmed path disclosure with informational severity, and a set of potential sensitive-resource exposures requiring further investigation. Two additional informational observations were recorded. 11

BASIC WEB VULNERABILITY ASSESSMENT

Internship Details

Name: Shaurya Gupta

Email: shauryagupta0106@gmail.com

Reference Number: CSN-OC-OL-26194

Registration Number: CSN-OC-2026-194

Domain: Cyber Security

Project Level: Beginner

Project Number: Project 4

Internship Month/Year: October 2026

Table of Contents

1. Introduction
2. Objectives
3. Scope and Authorization
4. Tools and Environment
5. Assessment Methodology
6. Implementation Details
7. Findings and Severity Classification
8. Risk and Impact Analysis
9. Mitigation Recommendations
10. Limitations
11. Summary and Conclusion
12. References
13. Appendix: Commands and Evidence

---

1. Introduction

A Basic Web Vulnerability Assessment is a preliminary security assessment performed to identify potentially unsafe configurations, information disclosures, and other weaknesses in a web application. It helps developers and security testers identify issues that should be investigated and addressed before an application is deployed in a production environment.

Web applications communicate with clients through HTTP requests and responses. Security-related response headers, access controls, exposed files, and publicly accessible application paths can influence the security of an application. Reviewing these elements can help identify weaknesses that may increase the risk of unauthorized access, information disclosure, or other attacks.

For this project, OWASP Juice Shop was deployed as a local training application in Docker on Kali Linux. The application was assessed using HTTP inspection with curl and automated scanning with Nikto. Burp Suite Community Edition was launched as part of the assessment workflow, but successful traffic interception was not confirmed in the available evidence.

The assessment identified missing recommended HTTP security headers, a publicly accessible robots.txt resource that disclosed a path for further investigation, and several additional paths flagged by Nikto that require manual verification.

The purpose of this report is to document the observed evidence, explain the potential security implications, and recommend practical mitigation measures without claiming that an unverified vulnerability was successfully exploited.

2. Objectives

The objectives of this project were:

- To deploy a local web application for authorized security testing.
- To inspect HTTP responses and identify the security headers returned by the application.
- To use Nikto to conduct a basic automated web server assessment.
- To inspect selected application endpoints using curl.
- To launch Burp Suite Community Edition and attempt to include proxy-based HTTP inspection in the assessment workflow.
- To distinguish confirmed observations from potential vulnerabilities and informational results.
- To classify findings according to their observed evidence and realistic impact.
- To document recommendations that can improve the application's security.
- To maintain an evidence-based assessment report suitable for a beginner-level cybersecurity internship project.

3. Scope and Authorization

Target: http://127.0.0.1:3000/

Application: OWASP Juice Shop

Environment: Local Kali Linux instance running OWASP Juice Shop in a Docker container.

Assessment period: 22 October 2026 to 23 October 2026. The main Nikto scan was recorded on 22 October 2026, followed by additional HTTP header inspection on 23 October 2026.

Authorization: Testing was restricted to the local OWASP Juice Shop training instance operated for this project.

Included activities

- Installing and checking the required assessment tools.
- Starting Docker and deploying OWASP Juice Shop locally.
- Confirming that the application responded over HTTP.
- Inspecting the homepage response headers.
- Saving the homepage response and HTTP header evidence.
- Running Nikto against the local target.
- Reviewing Nikto findings for missing headers, interesting paths, and possible information disclosure.
- Checking the HTTP responses from `/`, `/robots.txt`, and `/.well-known/security.txt`.
- Launching Burp Suite Community Edition to attempt proxy-based inspection.
- Documenting confirmed observations and potential issues.

Excluded activities

- Denial-of-service testing.
- Destructive testing.
- Testing systems outside the authorized local environment.
- Exploitation of real user accounts or production data.
- Password attacks or credential theft.
- Claims of successful exploitation without supporting evidence.
- Testing of endpoints on external systems not included in the project scope.

4. Tools and Environment

The following tools and environment details were recorded in the supplied terminal output.

Tool or component	Recorded version or configuration
Operating system	Kali Linux; exact release version not recorded
Web application	OWASP Juice Shop
Deployment	Docker container
Docker	28.5.2+dfsg4-5
Nikto	2.6.1
curl	8.22.0
Burp Suite	2026.9
Browser	Mozilla Firefox 153.4.0esr
Target address	`http://127.0.0.1:3000/`
Target port	TCP 3000
Docker container name	`juice-shop`
Docker image	`bkimminich/juice-shop:latest`

Tool purposes

Kali Linux: Provided the environment for executing the assessment commands and running the security tools.

Docker: Hosted the OWASP Juice Shop application locally. The recorded port mapping was `127.0.0.1:3000->3000/tcp`, restricting the published application port to the local loopback interface.

curl: Sent HTTP requests and recorded response headers and status codes for the target endpoints.

Nikto: Performed automated web server checks and identified missing recommended security headers and potentially interesting paths.

Burp Suite Community Edition: Was launched for proxy-based HTTP inspection. The available terminal evidence does not confirm that requests were successfully captured in HTTP history.

Firefox ESR: Was launched to open the local OWASP Juice Shop application.

5. Assessment Methodology

The assessment followed a basic, non-destructive workflow.
Step 1: Prepare the assessment environment
A project directory was created with separate folders for reports, screenshots, and results. Required tools were checked or installed through the Kali Linux package manager.
Step 2: Deploy the target application
Docker was enabled and started. The OWASP Juice Shop image was downloaded, and a container named `juice-shop` was started with port 3000 published on `127.0.0.1`.
The container status was checked using `sudo docker ps`.
Step 3: Confirm application availability
The command `curl -I http://127.0.0.1:3000/` was executed. The target returned `HTTP/1.1 200 OK`, confirming that the homepage was responding.
Step 4: Capture HTTP response evidence
The homepage response headers were saved to `results/response-headers.txt`, and the homepage body was saved to `results/homepage.html`.
Additional HTTP checks were conducted against:
- `/` - `/robots.txt` - `/.well-known/security.txt`
The outputs were saved in `results/http-checks.txt`.
Step 5: Run automated scanning
Nikto 2.6.1 was executed against the local target. Its output was saved to `results/nikto.txt` and copied to `results/nikto-scan-evidence.txt`.
The scan reported missing recommended headers, an entry in `robots.txt` pointing to `/ftp/`, and several paths that it considered potentially interesting.
Step 6: Review and classify findings
The response headers and Nikto output were reviewed to distinguish direct observations from scanner-generated alerts. Findings were classified as confirmed, potential, or informational according to the evidence available.
Step 7: Attempt proxy inspection
Burp Suite was launched twice. The recorded terminal output contains startup warnings and subsequent interruption, but no evidence that HTTP requests were successfully captured or analyzed through Burp Suite.
Therefore, Burp-based verification is recorded as incomplete.
Step 8: Prepare the report
The findings, evidence, limitations, and recommendations were documented in this report. No exploitation or data compromise is claimed.
6. Implementation Details
6.1 Project setup
The following project directory was created:
`/home/kali/SHAURYA_GUPTA/Project_4_Basic_Web_Vulnerability_Assessment`
It contained the following directories:
- `report/` - `screenshots/` - `results/`
A terminal session was also recorded in `commands.txt`.
6.2 Application deployment

The Docker service was started and enabled. The OWASP Juice Shop image was downloaded and launched using the following command:
`sudo docker run -d --name juice-shop -p 127.0.0.1:3000:3000 bkimminich/juice-shop`
The container was confirmed to be running using `sudo docker ps`.
The application was accessible at `http://127.0.0.1:3000/`.
6.3 HTTP header inspection
The homepage headers were inspected with:
`curl -I http://127.0.0.1:3000/`
The response included:
- `HTTP/1.1 200 OK` - `Access-Control-Allow-Origin: *` - `X-Content-Type-Options: nosniff` - `X-Frame-Options: SAMEORIGIN` - `Feature-Policy: payment 'self'` - `X-Recruiting: /#/jobs` - `Content-Type: text/html; charset=UTF-8`
The captured homepage response did not include `Content-Security-Policy`, `Permissions-Policy`, `Referrer-Policy`, or `Strict-Transport-Security`.
The full captured header output was saved to `results/response-headers.txt`.
6.4 Automated Nikto scan
The following command was executed:
`nikto -host http://127.0.0.1:3000/ -output results/nikto.txt`
The scan reported 8,826 requests, two errors, and 28 items. The recorded duration was approximately 115 seconds.
Notable results included:
- Missing recommended HTTP security headers. - An entry for `/ftp/` in `robots.txt`. - Potentially interesting paths such as `/.htpasswd`, `/.bash_history`, and several JSON filenames. - A possible administrative-interface signature at `/JAMonAdmin.jsp`.
These results were treated as scanner findings requiring interpretation and, where applicable, manual verification. The output alone does not establish that all flagged files contain sensitive information.
6.5 Manual HTTP checks
The following command was used to inspect the homepage response headers:
`curl -sS -D - -o /dev/null http://127.0.0.1:3000/`
The following command checked `robots.txt`:
`curl -sS -D - -o /dev/null http://127.0.0.1:3000/robots.txt`
The following command checked the security contact resource:
`curl -sS -D - -o /dev/null http://127.0.0.1:3000/.well-known/security.txt`
All three endpoints returned `HTTP/1.1 200 OK` in the recorded checks.
The combined output was saved in `results/http-checks.txt`.
6.6 Burp Suite
Burp Suite was launched from the terminal using:
`burpsuite`

Startup warnings were recorded, and the process was interrupted. A later launch was also interrupted. The supplied terminal log does not contain captured request entries or completed proxy analysis.
Burp Suite is therefore listed as an attempted tool, not as a source of verified vulnerability evidence.
6.7 Screenshot evidence
The supplied text file contains terminal logs but does not establish that screenshots were saved in the `screenshots/` directory. Screenshots should only be referenced in the final submission if the actual image files exist.
Recommended screenshots to capture before final submission are:
1. Docker container running with the local port mapping. 2. Homepage HTTP headers displayed in the terminal. 3. Nikto scan output showing the missing security headers. 4. Nikto scan output showing the `/ftp/` entry and potentially interesting paths. 5. Manual HTTP checks showing the status codes for the tested endpoints. 6. Burp Suite HTTP history, only if traffic capture is successfully completed.
7. Findings and Severity Classification
Finding F-01: Missing Recommended HTTP Security Headers
Affected endpoint: `http://127.0.0.1:3000/`
Severity: Low
Verification status: Confirmed configuration observation
Evidence:
The homepage returned `HTTP/1.1 200 OK`. The captured response included `X-Content-Type-Options: nosniff` and `X-Frame-Options: SAMEORIGIN`, but did not include the following recommended headers:
- `Content-Security-Policy` - `Permissions-Policy` - `Referrer-Policy` - `Strict-Transport-Security`
Nikto independently reported these headers as missing in its scan output.
Verification method:
The response was inspected using curl, and the result was compared with the corresponding Nikto output in `results/nikto.txt`.
Impact:
Missing security headers can reduce defense-in-depth protections. A suitable Content Security Policy can help restrict permitted resource sources, while Referrer Policy can control the referrer information sent with requests. Permissions Policy can limit access to selected browser features.
The missing headers do not independently prove that a specific attack is exploitable. HSTS is relevant to HTTPS deployments and cannot be meaningfully assessed as an enforced HTTPS policy from the local HTTP response alone.
Recommended mitigation:
Configure appropriate security headers for the intended deployment. Define a Content Security Policy that is compatible with the application's functionality, select an appropriate Referrer Policy, restrict unnecessary browser features through Permissions Policy, and configure HSTS for an HTTPS deployment.

Evidence reference: `results/response-headers.txt`, `results/http-checks.txt`, and `results/nikto.txt`.

Finding F-02: Publicly Accessible robots.txt Discloses a Path for Further Investigation

Affected endpoints:

- `http://127.0.0.1:3000/robots.txt`
- `http://127.0.0.1:3000/ftp/`

Severity: Informational

Verification status: Confirmed path disclosure; sensitive content exposure not confirmed

Evidence:

The `robots.txt` endpoint returned `HTTP/1.1 200 OK`. Nikto reported that `robots.txt` contained an entry for `/ftp/` and that the path returned a non-forbidden or redirect HTTP status of 200 during its scan.

Verification method:

The HTTP response for `/robots.txt` was checked using curl. The path disclosure was identified in the Nikto scan output. The available evidence does not include a detailed review of the content served by `/ftp/` or prove that sensitive files are exposed there.

Impact:

The `robots.txt` file can disclose paths that site owners prefer crawlers not to visit. This information may help an attacker identify areas for further reconnaissance. The file itself is not an access-control mechanism, and disclosure of a path does not establish unauthorized access to sensitive information.

Recommended mitigation:

Review the entries in `robots.txt` and remove unnecessary path disclosures where appropriate. Enforce authentication and authorization on sensitive resources, and ensure that the `/ftp/` path does not expose backups or confidential files. Retest the path and document its actual response after applying access controls.

Evidence reference: `results/http-checks.txt` and `results/nikto.txt`.

Finding F-03: Potential Exposure of Sensitive Files and Administrative Resources

Affected paths flagged by Nikto:

- `/.psql_history`
- `/.htpasswd`
- `/.bash_history`
- `/.mysql_history`
- `/.sh_history`
- `/.sqlite_history`
- `/userdata.json`
- `/login.json`
- `/master.json`
- `/masters.json`
- `/connections.json`
- `/connection.json`
- `/PasswordsData.json`
- `/users.json`
- `/conndb.json`
- `/conn.json`
- `/accounts.json`
- `/JAMonAdmin.jsp`

Severity: Medium, provisional pending manual verification

Verification status: Potential; not independently confirmed

Evidence:

Nikto reported these paths as potentially interesting. Its output included descriptions suggesting possible authorization information, shell-history exposure, database-related files, JSON resources, and a possible JAMon administrative interface.

The scan did not provide sufficient endpoint-by-endpoint response content in the supplied evidence to establish that each path exposes the resource described by the scanner.

Verification method:

The paths were identified by the automated Nikto scan saved in `results/nikto.txt`. The available evidence does not independently establish the content, sensitivity, or functionality of each flagged resource.

Manual verification is required to record each path's actual HTTP status and response content before confirming a vulnerability.

Impact:

If genuine sensitive files are accessible, they could expose credentials, account information, configuration details, database information, or shell history. An accessible administrative interface may create additional risk if it is functional and lacks appropriate access controls.

These impacts are conditional. The current evidence does not demonstrate that

credentials were retrieved, sensitive records were disclosed, or an administrative interface was successfully accessed.

Recommended mitigation:

Review every flagged path individually. Remove sensitive files from the web root, deny public access to configuration and backup files, store secrets outside web-accessible directories, and protect administrative interfaces with appropriate authentication and authorization. Retest each endpoint and document the actual status code and response content.

Evidence reference: `results/nikto.txt` and `results/nikto-scan-evidence.txt`.

Informational observation: Access-Control-Allow-Origin Header

Affected endpoint: `http://127.0.0.1:3000/`

Severity: Informational

Verification status: Header presence confirmed; vulnerability not established

Evidence:

The homepage response included:

```
Access-Control-Allow-Origin: *
```

Nikto also reported the wildcard value.

Impact:

A wildcard CORS response header allows cross-origin browser access to resources where the applicable browser CORS rules permit it. The presence of this header alone does not establish a vulnerability. The actual risk depends on the resources exposed, the other CORS response headers, authentication requirements, and whether sensitive data can be accessed cross-origin.

Recommended mitigation:

Review whether cross-origin access is required. If it is not required, remove the permissive policy. If it is required, configure the policy to allow only the intended origins where appropriate and test access to sensitive endpoints.

Evidence reference: `results/response-headers.txt` and `results/nikto.txt`.

Informational observation: Uncommon X-Recruiting Header

Affected endpoint: `http://127.0.0.1:3000/`

Severity: Informational

Verification status: Confirmed header presence; vulnerability not established

Evidence:

The homepage response included:

`X-Recruiting: /#/jobs`

Nikto identified this as an uncommon response header.

Impact:

The header reveals a route associated with the application's jobs page. No sensitive information disclosure or exploitable weakness was established by the supplied evidence.

Recommended mitigation:

Review whether this custom header is required. Remove it if it serves no operational purpose. Its presence alone does not justify classifying the application as vulnerable.

Evidence reference: `results/response-headers.txt` and `results/nikto.txt`.

8. Risk and Impact Analysis

The assessment identified one confirmed low-severity configuration observation, one confirmed path disclosure with informational severity, and a set of potential sensitive-resource exposures requiring further investigation. Two additional informational observations were recorded.

Finding	Status	Severity	Risk interpretation
F-01: Missing recommended security headers	Confirmed observation	Low	Reduced defense in depth; no exploit demonstrated
F-02: robots.txt path disclosure	Path disclosure confirmed	Informational	May assist reconnaissance; sensitive exposure not confirmed
F-03: Potential sensitive files and administrative resources	Potential	Medium, provisional	Impact could be significant if genuine sensitive resources are accessible
I-01: Wildcard CORS header	Header confirmed	Informational	Risk depends on resource exposure and CORS behavior
I-02: X-Recruiting header	Header confirmed	Informational	No security impact established

The potential sensitive-resource finding should receive manual verification priority because confirmed exposure of credentials or configuration files could have a greater impact than the other observations. However, the provisional severity must not be interpreted as proof that the resources are accessible or contain sensitive information.

Evidence file	Purpose
`commands.txt`	Recorded terminal session
`results/assessment-time.txt`	Assessment timestamp
`results/target.txt`	Target and scope
`results/response-headers.txt`	Captured homepage headers and response
`results/homepage.html`	Saved homepage body
`results/nikto.txt`	Nikto scan output
`results/nikto-scan-evidence.txt`	Copy of Nikto scan output
`results/http-checks.txt`	Combined HTTP response checks

The presence of individual screenshot files was not established by the supplied evidence. Any screenshots included in the final submission should be genuine captures of the actual commands, tool outputs, and verification results.

End of Report